PYTHON PROGRAMMING / WEEK 12 / SESSION 1

디버깅과 회귀 테스트

동작하는 MVP에서 수정의 근거를 찾기

120분, 휴식 포함 / 담당: MinGi KYUNG

한국어판

학습 목표

- 고정된 작은 입력으로 실패를 재현한다.
- traceback을 읽고 변수 상태를 확인한다.
- 회귀 테스트를 작성하고 원인을 수정한다.
- 수정이 요구사항을 만족하는 이유를 설명한다.

11주차 MVP 복습

- 시장 보고서는 CSV를 읽고 종목을 분석한 뒤 HTML을 저장한다.
- 종목 코드는 문자열로 유지하고 계산 전에 가격을 검증한다.
- 파일 누락은 오류, 헤더만 있는 정상 파일은 빈 데이터다.
- 실행에 실패해도 기존 보고서는 보존해야 한다.

확인: 기능의 완료 기준에는 성공과 실패 시의 동작이 모두 포함된다.

디버깅의 반복 과정

- 재현: 같은 명령과 같은 입력으로 같은 실패를 확인한다.
- 진단: 함수 계약, traceback, 변수 상태를 비교한다.
- 기록: 관찰한 결함을 검출하는 실패 테스트를 만든다.
- 수정: 실패 원인을 설명하는 최소 부분을 고친다.
- 재검증: 해당 테스트를 먼저, 전체 테스트를 나중에 실행한다.

예시: BUG-01의 입력 100과 110을 고정한다. 수정한 식과 테스트가 모두 +10%라는 계약을 만족하는지 확인한다.

오류의 세 가지 유형

- 문법 오류: 파이썬이 코드를 해석할 수 없다.
- 실행 중 예외: 실행 도중 어떤 연산이 실패한다.
- 논리 버그: 실행은 되지만 잘못된 결과를 반환한다.
- 테스트 실패의 원인은 코드·데이터·테스트 중 어디에나 있을 수 있다.

```
100 -> 110 should be +10%
A result near +9.09% is a logic bug.
```

버그 보고서

- 실행 명령, 환경, 정확한 입력을 기록한다.
- 기대 동작과 실제 동작을 구분한다.
- 기대값은 구현 결과가 아니라 요구사항에서 구한다.
- 실습에는 수업용 가상 데이터를 사용한다.

```
BUG-01 / daily percentage change
Input: previous_close=100, close=110
Expected: 10.0
Observed: 9.090909...
```

최소 재현 입력

- 첫 실험에서는 CSV·브라우저·보고서 생성을 제외한다.
- 정수 두 개로 순수 함수를 직접 호출한다.
- 조사하는 동안 실패 입력을 고정한다.

```
cd 12week/examples/debug_lab
python3 -c "from market_rules import change_pct; \
print(change_pct(100, 110))"
```

Traceback 읽기

- 마지막 줄에서 예외 유형과 메시지를 먼저 확인한다.
- 자신의 코드에서 실패한 마지막 연산을 찾는다.
- 호출자를 거슬러 올라가 값이 전달된 경로를 확인한다.
- 실패 위치와 근본 원인은 다를 수 있다.

```
File "market_rules.py", ... in summarize
    ... sum(changes) / len(changes)
ZeroDivisionError: division by zero
```

두 호출에 걸친 Traceback 해석

```
Traceback (most recent call last):
  File "debug_empty.py", line 2, in <module>
    summarize([])
  File "market_rules.py", line 18, in summarize
    return {"count": len(changes),
            "mean": sum(changes) / len(changes)}
ZeroDivisionError: division by zero
```

- 마지막 줄: 실패한 연산은 0으로 나누기다.
- 함수 프레임: changes가 비어 있어 len(changes)는 0이다.
- 호출자 프레임: 재현 스크립트가 빈 목록을 전달했다.
- 계약: 빈 목록은 유효한 입력이다. 요약 함수를 고치고 이 상태를 테스트한다.

가설과 확인

- 관찰: 계산된 상승률이 기대값보다 작다.
- 가설: 이전 가격 대신 현재 가격으로 나누고 있다.
- 분자와 분모를 각각 확인한다.
- 직접 계산한 (110 - 100) / 100 * 100 = 10과 비교한다.

원칙: 증거로 원인을 확인한 뒤 한 가지씩 수정한다.

print로 상태 확인

임시 출력으로 식에 들어가는 값을 확인할 수 있다.

```
previous_close, close = 100, 110
print("difference:", close - previous_close)
print("previous:", previous_close, "current:", close)
```

- 값과 함께 자료형도 확인한다.
- 수정 후에는 임시 디버깅 출력을 제거한다.

pdb로 한 줄씩 실행

터미널에서 중단점을 설정하고 실패한 호출의 상태를 확인한다.

```
from market_rules import summarize
summarize([])
```

```
python3 -m pdb debug_empty.py
```

- 아래 코드를 debug_empty.py로 저장한 뒤 명령을 실행한다.
- n: 다음 줄 실행, s: 함수 안으로 진입.
- 함수 안에서 p changes, p len(changes)로 상태 확인.
- q: 종료. VS Code에서는 함수 return 줄에 중단점을 둔다.

pdb 실행 기록 따라 읽기

앞 슬라이드의 debug_empty.py를 사용한다. 실제 줄 번호는 다를 수 있다.

```
(Pdb) n
-> summarize([])
(Pdb) s
> market_rules.py(...)summarize()
(Pdb) p changes
[]
(Pdb) p len(changes)
0
```

- n으로 import를 실행하면 함수 호출 직전에서 멈춘다.
- s로 summarize에 진입하고 본문 실행 전에 인자를 확인한다.
- 나눗셈을 계속 실행하면 예외가 재현된다. q로 종료한다.

함수 계약과 테스트 사례

계약	입력	기대 결과
이전 가격 기준 변화율	100 → 110	+10%
가격 하락	100 → 90	-10%
가격 유지	100 → 100	0%
양의 가격만 허용	0 → 100	ValueError
빈 요약	[]	count 0, mean None

기대값을 독립적으로 계산할 수 있는 입력을 선택한다.

정상·경계·오류 입력 나누기

분류	입력	기대 결과
최소 유효 가격	가격 1, 2	+100%
경계 밖의 가격	가격 0, 100	ValueError
잘못된 자료형	가격 True, 100	ValueError
빈 요약	[]	평균 None
관측값 한 개	[5]	평균 5

- 가격 0과 1처럼 허용 여부가 바뀌는 지점을 선택한다.
- bool은 int의 하위 클래스이므로 True도 별도 사례로 확인한다.

unittest로 검증하기

```
import unittest
from market_rules import change_pct

class ChangeTests(unittest.TestCase):
    def test_increase(self):
        self.assertAlmostEqual(change_pct(100, 110), 10.0)
```

- 입력 준비, 함수 호출, 결과 단언 순서로 작성한다.
- 부동소수점 계산 결과에는 근사 비교를 사용한다.
- 기본 자동 탐색은 test*.py 이름의 파일을 찾는다.

입력 준비·실행·결과 단언

unittest.TestCase를 상속한 클래스 안에 작성하는 메서드다.

```
def test_increase(self):
    # Arrange: known input and expected answer.
    previous, close = 100, 110
    expected = 10.0
    # Act: call the function once.
    actual = change_pct(previous, close)
    # Assert: check the contract.
    self.assertAlmostEqual(actual, expected)
```

- 기대값 10은 손으로 계산해 먼저 정한다.
- 구현의 계산식을 기대값에 복사하면 같은 결함을 함께 복사할 수 있다.

계약에 맞는 assertion 선택

assertion	예시	확인할 계약
assertEqual	count, 3	정확한 개수·순서
assertAlmostEqual	change, 10.0	계산한 실수
assertIsNone	mean	평균 없음
assertRaises	ValueError	잘못된 입력 거절

```
with self.assertRaises(ValueError):
    change_pct(0, 100)
```

- with 블록 안에 호출을 넣는다. ValueError가 없거나 다른 예외가 발생하면 실패한다.

실패·오류·회귀

- 실패(Failure): assertion이 기대값과 실제값의 차이를 발견했다.
- 오류(Error): 예상하지 않은 예외로 테스트가 중단됐다.
- 회귀(Regression): 변경 후 기존에 되던 동작이 깨졌다.
- 수정 전에 새 테스트가 해당 결함을 발견하는지 확인한다.
- 수정 후 전체 테스트로 다른 동작의 손상을 확인한다.

```
python3 -m unittest -v
```

테스트 하나 실행하고 결과 읽기

12week/examples/debug_lab 폴더에서 실행한다.

```
python3 -m unittest -v test_market_rules.MarketRuleTests.test_empty_summary
python3 -m unittest -v
```

```
Ran 5 tests ...
FAILED (failures=3, errors=1)
```

- 시작 코드의 결과다. 결함 세 개가 문제 네 건으로 보고된다.
- 가격 테스트 메서드 안의 subTest 두 개가 각각 실패하기 때문이다.
- 한 문제를 집중 수정한 뒤 전체를 실행한다. 테스트 개수만으로 충분한 검증을 판단할 수 없다.

휴식 — 10분

쉬기 전에 실패한 테스트 하나와 해당 계약을 확인한다.

휴식 후 실습 코드의 의도적인 결함 세 개를 수정한다.

실습 함수의 계약

- change_pct: 양의 정수 가격만 허용하며 bool은 제외한다. 반올림하지 않은 등락률을 반환한다.
- summarize: 검증된 숫자 목록에서 개수와 평균을 구한다. 빈 목록의 평균은 None이다.
- rank_changes: 검증된 행을 등락률 내림차순·코드 오름차순으로 정렬한다. 새 목록을 반환하고 입력을 바꾸지 않는다.
- 이 실습에서는 계산 규칙을 파일 입출력과 분리한다.

실습 A: 등락률 계산

- 수정 전에 100 → 110의 잘못된 결과를 재현한다.
- 분모를 확인하고 원인을 기록한다.
- 식을 수정한 뒤 상승·하락·보합 테스트를 실행한다.
- 반올림은 계산 함수가 아니라 출력 단계에서 처리한다.

질문: 이 테스트가 잘못된 식을 검출하는가, 그대로 반복하는가?

회귀 테스트로 수정 전후 비교

```
# Before: wrong reference price.
return (close - previous_close) / close * 100
# After: change relative to the previous price.
return (close - previous_close) / previous_close * 100
```

- 수정 전: 100에서 110의 결과가 약 9.09라 테스트가 실패한다.
- 수정 후: 같은 테스트에서 10을 반환하여 통과한다.
- 100에서 90으로 하락하면 -10%인지도 확인한다.
- 기존 입력 검증과 공개 함수의 인자 형식은 유지한다.

확인: 상수 10을 반환하면 한 예제는 통과해도 하락·보합 사례에서 실패한다.

실습 B: 빈 요약

- summarize([])를 호출하고 traceback을 읽는다.
- 관측값이 없으면 산술평균도 없다.
- 개수는 0, 평균은 None으로 반환한다.
- 수정 후 원소 한 개와 여러 개의 경우도 확인한다.
- 실제 평균 0과 데이터 없음은 구분할 수 있어야 한다.

데이터 없음과 실제 평균 0

```
summary = {"count": 3, "mean": 0.0}
mean = summary["mean"]
label = "N/A" if mean is None else f"{mean:.2f}%"
print(label)  # 0.00%
```

- []의 개수는 0이고 평균은 None이다.
- [10, -10, 0]의 개수는 3이고 실제 평균은 0이다.
- if not mean은 None과 0.0을 모두 거짓으로 판단한다.
- 값이 없는 상태를 표시할 때는 None을 명시적으로 확인한다.

실습 C: 동률 정렬

- 등락률은 내림차순으로 정렬한다.
- 등락률이 같으면 종목 코드는 오름차순으로 정렬한다.
- 기대 순서와 다른 순서로 입력을 준비한다.
- 원래 목록과 행의 값이 바뀌지 않았는지 확인한다.

```
Input codes:  000003(-1), 000002(+2), 000001(+2)
Output codes: 000001, 000002, 000003
```

두 항목으로 구성한 정렬 키

```
key = lambda row: (-row["change_pct"], row["code"])
```

코드와 등락률	정렬 키	최종 위치
000003, -1	(1, "000003")	3
000002, +2	(-2, "000002")	2
000001, +2	(-2, "000001")	1

- 파이썬은 튜플의 앞 항목부터 오름차순으로 비교한다.
- 등락률에 음수를 붙이면 원래 큰 값이 먼저 온다.
- 첫 항목이 같을 때 두 번째 항목인 코드가 순서를 결정한다.

새 목록과 공유되는 행 객체

```
before = [dict(row) for row in rows]
ranked = rank_changes(rows)
self.assertEqual(rows, before)
self.assertIsNot(ranked, rows)
```

- rows.sort는 입력 목록을 바꾼다. sorted는 새 목록을 만든다.
- 새 목록 안의 딕셔너리는 여전히 원래 행 객체를 참조한다.
- 실습의 계약은 정렬 중 원본을 바꾸지 않는 것이다. 깊은 복사를 보장하지 않는다.
- 평평한 각 행을 복사해 비교용 상태를 만든다. before = rows는 같은 객체를 가리켜 변경을 놓친다.

추가 테스트 두 개

- 제공된 테스트에 없는 사례를 선택한다.
- 예: 최소 양의 가격, 원소 한 개의 요약, 빈 순위.
- 함수를 실행하기 전에 기대값을 먼저 적는다.
- 각 사례가 확인하는 요구사항을 기록한다.
- 마지막 수정 후 전체 테스트를 다시 실행한다.

약한 검사와 결함을 검출하는 검사

```
# Weak: an empty dictionary would also pass.
self.assertIsNotNone(summarize([]))
# Stronger: verify the agreed empty-state values.
self.assertEqual(summarize([]), {"count": 0, "mean": None})
```

- 실제 구현을 그럴듯한 오답으로 바꾸면 테스트가 실패해야 한다.
- 오름차순으로 정렬하거나 동률 처리를 빼면 검출할 수 있는지 묻는다.
- 이런 실수가 드러나도록 기대 순서와 반대인 입력도 준비한다.
- 기대값은 검사 대상 함수를 호출해서 만들지 않는다.

수정 내용 리뷰

- 실패 입력과 관련 코드 diff를 보여준다.
- 값이나 제어 흐름을 근거로 원인을 설명한다.
- 테스트를 삭제하거나 검증을 약하게 바꾸지 않았는지 확인한다.
- 다른 유효 입력에서도 계약이 깨질 가능성을 검토한다.
- 실습 폴더의 debugging-report.md에 리뷰를 기록한다.

제출과 확인 질문

- 수정 코드, 추가 테스트 두 개, 디버깅 보고서를 제출한다.
- 실행 명령과 실제 테스트 요약을 포함한다.
- 빈 평균과 평균 0은 왜 다른가?
- 동률 종목을 코드 역순으로 입력하는 이유는 무엇인가?
- 다음 수업: 입력 검증, 파일 실패, 기능 확장 테스트.